

Documentación de Middlewares: Integración del Frontend, Procesos y Negocio de la API

Esta documentación detalla la arquitectura de middlewares en el ecosistema de la API (`src/api/middleware/` y adjuntos globales), las pautas para que el frontend interactúe con estas barreras de seguridad, las excepciones de desarrollo y las reglas del flujo de negocio que gobiernan la validación de peticiones HTTP y WebSockets.

Sección 1: Documentación para el Frontend (Guía de Integración)

Los middlewares actúan como interceptores silenciosos de las peticiones. Para que el frontend pueda consumir los recursos de la API sin interrupciones, debe alinearse con las siguientes especificaciones técnicas.

1. Cabecera y Autenticación JWT

La gran mayoría de las rutas protegidas requieren de autenticación. El frontend debe inyectar la cabecera HTTP estándar en cada petición.

- Formato Requerido:

```
Authorization: Bearer <JWT_TOKEN>
```

- Códigos de Error Comunes y Mensajes:** El frontend debe estar programado para interceptar respuestas con código de estado HTTP `401 Unauthorized` y actuar según el mensaje recibido:

Mensaje en JSON (<code>message</code>)	Causa	Acción recomendada en Frontend
"No autorizado - Token faltante"	Falta la cabecera <code>Authorization</code> o no tiene el formato <code>Bearer</code> .	Redirigir al inicio o forzar inicio de sesión.
"Sesión expirada"	El servicio de identidad OAuth devolvió un error de validez (token caducado).	Limpiar almacenamiento de sesión y redirigir al login.
"No autenticado"	Hubo un error de conexión o validación general con el proveedor de identidades.	Mostrar banner de error o forzar reconexión.

2. Token de Bypass para Desarrollo (Mock Token)

Para agilizar el desarrollo de componentes del frontend en entornos locales (donde configurar el servicio OAuth completo puede ser complejo o innecesario), el middleware de autenticación incluye un **Bypass de Desarrollo**.

- **Token Mock Autorizado:** `dev-mock-token-xyz-123`
- **Condición:** El servidor debe ejecutarse con `NODE_ENV=development`.
- **Efecto:** Si el frontend envía `Authorization: Bearer dev-mock-token-xyz-123`, el middleware saltará la validación externa y creará un usuario simulado con permisos administrativos:

```
{
  "id": "mock-id-123",
  "name": "Usuario Desarrollo",
  "email": "dev@cicsa.com",
  "role": "admin"
}
```

3. Autenticación en la Conexión de WebSockets (Socket.IO Handshake)

Al igual que en las peticiones REST, la conexión a WebSockets está interceptada por un middleware de autenticación en tiempo de conexión (handshake).

- **Parámetros de Handshake obligatorios:** Al conectar con el cliente de `socket.io`, el frontend debe enviar las credenciales en la propiedad `auth`:

```
const socket = io("http://localhost:3007", {
  auth: {
    officeId: "MTY",           // Identificador de sucursal/oficina (Obligatorio)
    profile: "Operador",      // Perfil del usuario
    type: "web"               // 'web' o 'scanner' (Electron)
  }
});
```

- **Respuesta ante falla:** Si no se provee `officeId`, el middleware de conexión rechaza el enlace emitiendo una excepción: `CREDENCIALES_INVALIDAS: Se requiere officeId`.

4. Cross-Origin Resource Sharing (CORS) y Límites de Body

- **CORS:** La API restringe las peticiones cruzadas basándose en el entorno. En desarrollo, permite cualquier origen (`true`). En producción, restringe las solicitudes basándose en el dominio exacto guardado en la variable de entorno `ALLOWED_ORIGIN`.
- **Métodos Permitidos:** `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `OPTIONS`.
- **Cabeceras Permitidas:** `Content-Type`, `Authorization`, `X-Is-Development`, `Accept`.
- **Límites de Carga (Body Limits):** El body parser de Express está limitado a **100 MB** (`express.json({ limit: '100mb' })`). Esto es crucial para que el frontend pueda subir imágenes digitalizadas en alta definición o chunks masivos de Excel sin provocar un error `413 Payload Too Large`.

5. Estructura Estándar de Errores (Centralized Error Payload)

Cualquier excepción no controlada dentro del backend es atrapada por el middleware de error centralizado. El frontend recibirá siempre el estado `500 Internal Server Error` con el siguiente formato JSON legible:

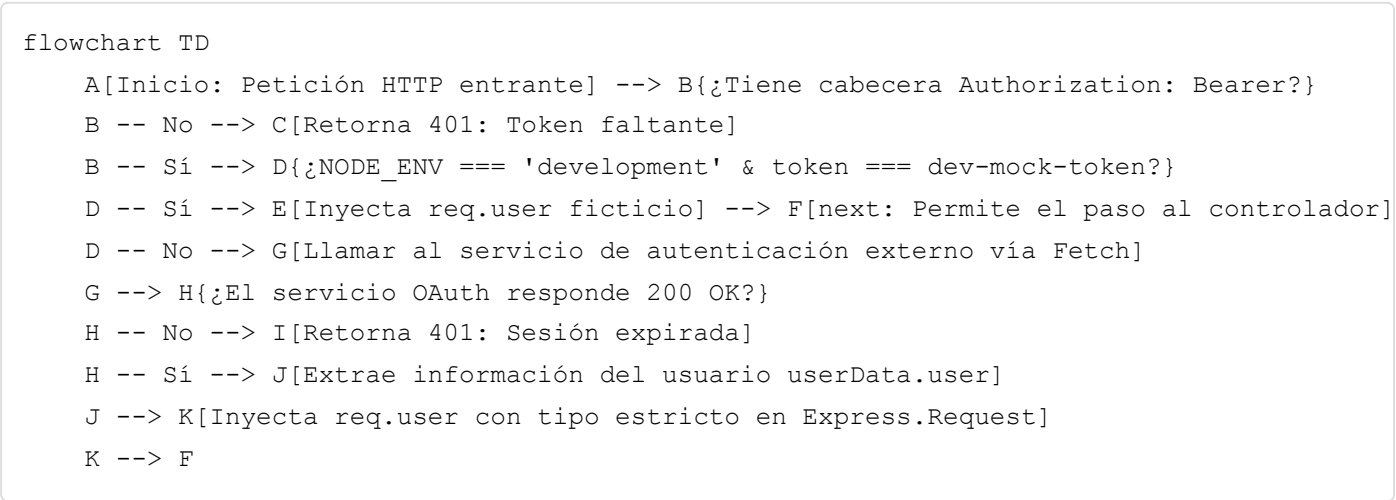
```
{
  "error": true,
  "message": "Descripción detallada del error de base de datos o lógica."
}
```

Sección 2: Guía de Procesos y Negocio de Middlewares

Esta sección detalla los flujos lógicos internos y las responsabilidades del negocio de cada middleware del sistema.

1. Middleware de Autenticación (`auth.middleware.ts`)

- **Propósito de Negocio:** Validar la identidad de los usuarios del sistema de costeo antes de que realicen cualquier operación que modifique partidas, presupuestos o cargue reportes diarios.
- **Flujo del Proceso Interno:**



- **Detalles Técnicos:** El middleware realiza un `fetch` hacia `${process.env.AUTH_SERVICE_URL}/api/auth/oauth2/userinfo` . El usuario validado se asigna a la propiedad `req.user` , cuya interfaz está extendida de forma global en `src/types/express.d.ts` a partir del tipo `User` de `src/utils/typesInterfaces.ts` .

2. Middleware del Handshake de WebSockets (`socket.manager.ts`)

- **Propósito de Negocio:** Segregar las conexiones de los clientes en salas (`rooms`) específicas basadas en su sucursal de negocio (`officeId`) para que las alertas del scanner de reportes diarios solo lleguen a los operadores correspondientes.
- **Flujo del Proceso Interno:**
 1. Intercepta el inicio de conexión de Socket.IO.

2. Inspecciona la propiedad `socket.handshake.auth`.
 3. Si no existe la propiedad o no tiene `officeId`, bloquea la conexión ejecutando `next(new Error())`.
 4. Si se valida la cabecera de conexión, almacena el `officeId`, `profile` y `clientType` en el objeto de datos del socket (`socket.data`).
 5. Ejecuta `next()` para dar paso a la asignación de la sala `room_${officeId}` y registrar la conexión en los logs o el mapa de conexiones activas (`scanners`).
-

3. Middlewares de Express Globales (`src/app.ts`)

- **CORS Middleware:** Configura las políticas de seguridad de origen. Para entornos locales desactiva la restricción; en producción, restringe a la dirección configurada de producción para evitar ataques de Cross-Site Scripting (XSS).
 - **JSON and Urlencoded Parser:** Interceptores de bajo nivel de Express que formatean el flujo de datos entrante (Stream) a objetos JSON y parámetros codificados en URL. Poseen un límite de 100 MB.
 - **Morgan Logger:** Registra en consola todas las llamadas HTTP indicando método, ruta, código de respuesta y tiempo de procesamiento en milisegundos.
 - **Static Middleware:** Expone la ruta `/static` mapeándola directamente al directorio local `scanned_images/` para permitir que el frontend o servicios externos puedan visualizar los reportes PDF cortados.
-

4. Middleware Central de Control de Errores (`src/app.ts` - línea 45)

- **Propósito de Negocio:** Garantizar la estabilidad del servidor ante excepciones no controladas en las capas de controladores y servicios (como fallas de conexión a base de datos o lecturas nulas), ocultando trazas de código sensibles en entornos de producción.
- **Flujo del Proceso Interno:**
 1. Atrapa los errores lanzados o pasados mediante la función `next(error)` en cualquier controlador de la API.
 2. Registra la traza del error en la consola del servidor.
 3. Devuelve al frontend una respuesta HTTP estructurada con código de estado `500` y el mensaje de error procesado, evitando que el servidor Express caiga o detenga sus procesos en segundo plano.